

Jardin de Charlotte

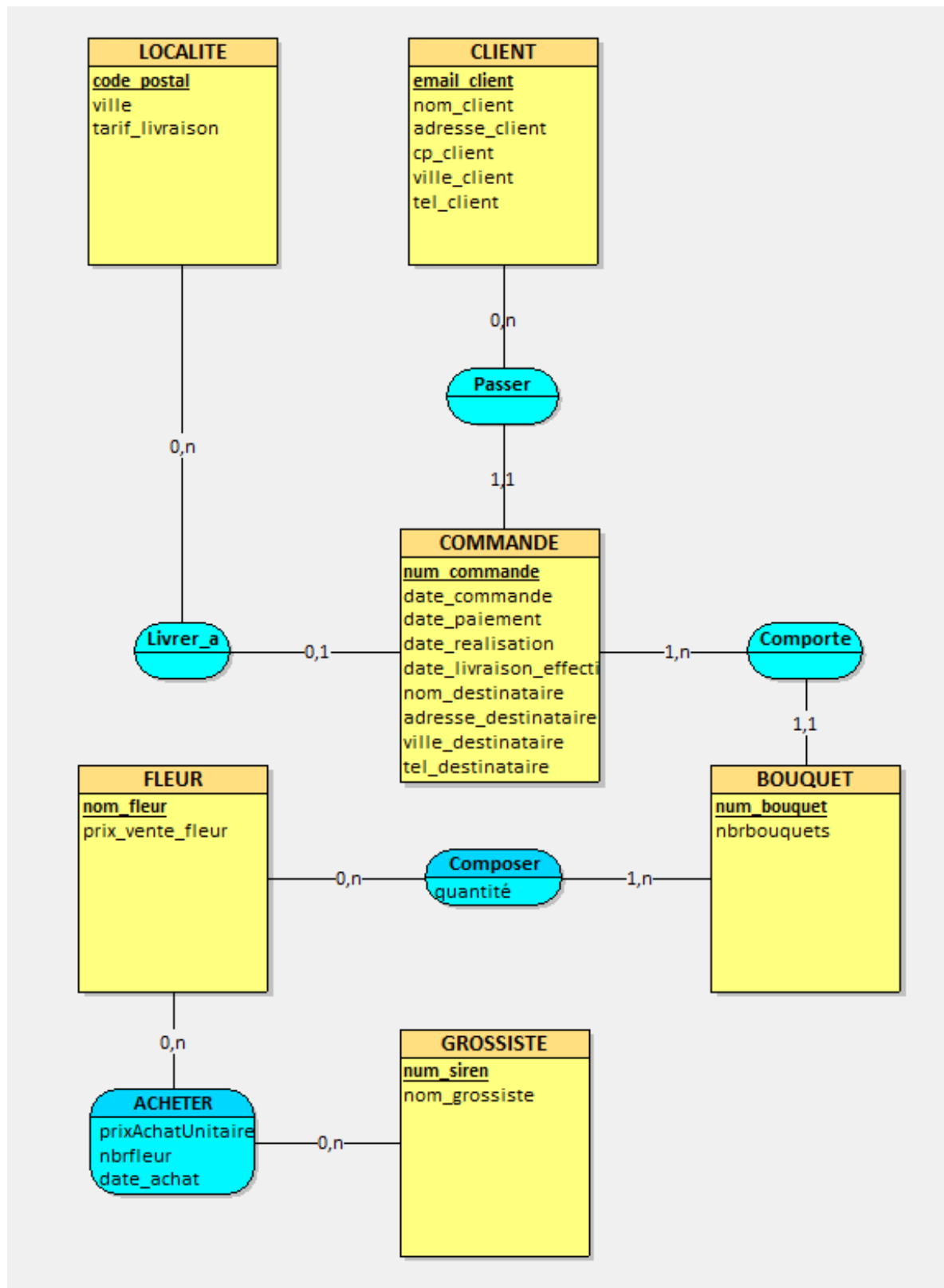
SAE : Conception et Création d'une Base de Données

Table des matières

Partie 1 : Conception	3
Modèle Conceptuel de Données (MCD)	3
Dictionnaire des Données	4
Entité COMMANDE	4
Entité CLIENT	5
Entité LOCALITE	5
Entité FLEUR	5
Entité GROSSISTE	6
Entité BOUQUET	6
Propriétés de l'Association COMPOSER	6
Propriétés de l'Association ACHETER	6
Partie 2 : Implémentation SQL	7
Création de la Base de Données(Q4)	7
Gestion des référentiels (Localités, Grossistes, Fleurs)	7
Gestion des Clients et Commandes	8
Détail des produits (Bouquets et Compositions)	9
Insertion des Données(Q5)	11
Exploitation des Données(Q7)	13
R1. Le nombre de commandes payées en 2021	13
R2. Les 3 premiers codes postaux où il y a le plus de livraisons	13
R3. La fleur la plus utilisée en 2021	14
R4. Le nombre moyen de bouquets par client	14*+
R5. Les fleurs jamais utilisées dans les bouquets	15
R6. La fleur préférée par client	15
R7. La liste des numéros de commande associés à la date, au nom du client et au montant ...	16
R8. Le nom du client qui a passé la commande la plus chère avec retrait en magasin	16
R9. Le nom du client qui a passé la commande la plus chère (Livraison ou Retrait)	17
Lien vers les codes utilisé	17

Partie 1 : Conception

Modèle Conceptuel de Données (MCD)



Dictionnaire des Données

Entité COMMANDE

L'entité COMMANDE centralise les données relatives à la transaction, incluant les dates clés (commande, paiement, réalisation) et les coordonnées spécifiques de livraison.

Code (MCD)	Libellé / Description	Type
numCommande	Numéro unique de la commande (Identifiant)	Entier
dateCommande	Date de passage de la commande	Date
datePaiement	Date du règlement	Date
dateRealisation	Date de confection du bouquet	Date
dateLivraison	Date de livraison effective demandée	Date
nomDestinataire	Nom du destinataire (si différent du client)	Texte
telDestinataire	Téléphone du destinataire	Texte
adrDestinataire	Adresse de livraison	Texte
villeDestinataire	Ville de livraison	Texte

Entité CLIENT

L'entité CLIENT recense les informations personnelles nécessaires à l'identification et à la facturation du donneur d'ordre.

Code (MCD)	Libellé / Description	Type
mailClient	Adresse email du client (Identifiant)	Texte
nomClient	Nom de famille du client	Texte
numTelClient	Numéro de téléphone du client	Texte
adresseClient	Adresse de facturation du client	Texte
villeClient	Ville du client	Texte
cpClient	Code postal du client	Texte

Entité LOCALITE

L'entité LOCALITE sert de référentiel géographique pour déterminer automatiquement les frais de port en fonction du code postal de livraison.

Code (MCD)	Libellé / Description	Type
codePostal	Code postal de la zone de livraison (Identifiant)	Texte
tarifLivraison	Prix de la livraison pour cette zone	Monétaire

Entité FLEUR

L'entité FLEUR constitue le catalogue des variétés disponibles à la vente au détail, associées à leur prix unitaire actuel.

Code (MCD)	Libellé / Description	Type
nomFleur	Nom de la variété de fleur (Identifiant)	Texte
tarifFleur	Prix de vente unitaire de la fleur	Monétaire

Entité GROSSISTE

L'entité GROSSISTE identifie les entreprises fournisseurs auprès desquelles le magasin s'approvisionne en matières premières.

Code (MCD)	Libellé / Description	Type
numSIREN	Numéro SIREN du grossiste (Identifiant)	Texte
nomGrossiste	Nom de l'entreprise grossiste	Texte

Entité BOUQUET

L'entité BOUQUET représente chaque article spécifique au sein d'une commande. Il s'agit d'une entité faible identifiée relativement à la commande.

Code (MCD)	Libellé / Description	Type
idBouquet	Numéro séquentiel du bouquet dans la commande	Entier
nbrBouquets	Quantité de ce type de bouquet commandée	Entier

Propriétés de l'Association COMPOSER

Cette structure détaille la composition florale d'un bouquet en précisant la quantité nécessaire pour chaque variété de fleur.

Code (MCD)	Libellé / Description	Type
quantiteFleur	Nombre de tiges d'une fleur dans un bouquet	Entier

Propriétés de l'Association ACHETER

Cette structure permet de conserver l'historique des approvisionnements, liant un grossiste à une fleur pour une date et un prix d'achat donnés.

Code (MCD)	Libellé / Description	Type
dateAchat	Date d'achat au grossiste	Date
prixAchatUnit	Prix d'achat unitaire négocié ce jour-là	Monétaire
nbrFleursAchat	Nombre de fleurs achetées au grossiste	Entier

Partie 2 : Implémentation SQL

Création de la Base de Données(Q4)

Gestion des référentiels (Localités, Grossistes, Fleurs)

Création de la table LOCALITE

Table LOCALITE Cette table matérialise l'entité **LOCALITE** servant à référencer les zones de livraison. Nous avons inclus une contrainte d'intégrité sur le tarif de livraison ($\text{tarifLivraison} \geq 0$) : celui-ci doit être positif ou nul (le cas nul correspondant à une offre de livraison gratuite), mais jamais négatif.

```
1 CREATE TABLE LOCALITE (  
2     codePostal VARCHAR2(5),  
3     ville VARCHAR2(50),  
4     tarifLivraison NUMBER(5,2) NOT NULL,  
5     CONSTRAINT pk_localite PRIMARY KEY (codePostal),  
6     CONSTRAINT ck_tarif_liv_pos CHECK (tarifLivraison >= 0)  
7 );  
8
```

Création de la table FLEUR

Table FLEUR Cette table correspond directement à l'entité **FLEUR** du MCD. Une contrainte de vérification (CHECK) a été ajoutée sur le tarif ($\text{tarifFleur} \geq 0$) pour garantir qu'aucune fleur ne puisse être enregistrée avec un prix de vente négatif, assurant ainsi la cohérence économique du catalogue.

```
1 CREATE TABLE FLEUR (  
2     nomFleur VARCHAR2(50),  
3     tarifFleur NUMBER(5,2) NOT NULL,  
4     CONSTRAINT pk_fleur PRIMARY KEY (nomFleur),  
5     CONSTRAINT ck_tarif_fleur_pos CHECK (tarifFleur >= 0)  
6 );
```

Création de la table GROSSISTE

Table GROSSISTE Cette table représente l'entité **GROSSISTE**. Elle ne comporte pas de contrainte CHECK spécifique autre que la clé primaire (numSIREN), car les données sont ici purement informatives (nom du fournisseur).



```
1 CREATE TABLE GROSSISTE (  
2     numSIREN VARCHAR2(20),  
3     nom VARCHAR2(50),  
4     CONSTRAINT pk_grossiste PRIMARY KEY (numSIREN)  
5 );
```

Gestion des Clients et Commandes

Création de la table CLIENT

Table CLIENT Cette table correspond à l'entité **CLIENT**. L'adresse email (mailClient) a été choisie comme clé primaire, garantissant l'unicité de chaque client dans la base, conformément aux règles de gestion implicites des données fournies.



```
1 CREATE TABLE CLIENT (  
2     mailClient VARCHAR2(150),  
3     nomClient VARCHAR2(100) NOT NULL,  
4     adresseClient VARCHAR2(150),  
5     codePostalClient VARCHAR2(5),  
6     villeClient VARCHAR2(50),  
7     numTelClient VARCHAR2(20),  
8     CONSTRAINT pk_client PRIMARY KEY (mailClient)  
9 );  
10
```

Création de la table COMMANDE

Table COMMANDE Cette table traduit l'entité **COMMANDE** et matérialise les associations **PASSER** (clé étrangère vers Client) et **LIVRER_A** (clé étrangère vers Localité). Plusieurs contraintes de cohérence chronologique ont été ajoutées pour valider les étapes de la commande comme la date de livraison effective et la date de réalisation qui doivent être **postérieures ou égales** à la date de la commande et la date de paiement, si renseignée, ne peut pas être antérieure à la prise de commande

```
1 CREATE TABLE COMMANDE (  
2     numCommande VARCHAR2(5),  
3     mailClient VARCHAR2(150),  
4     dateCommande DATE NOT NULL,  
5     datePaiement DATE,  
6     dateRealisation DATE,  
7     dateLivraisonEffective DATE,  
8     nomDestinataire VARCHAR2(100),  
9     villeDestinataire varchar(50),  
10    adresseDestinataire VARCHAR2(150),  
11    telDestinataire VARCHAR2(20),  
12    codePostalLivraison VARCHAR2(5),  
13    CONSTRAINT pk_commande PRIMARY KEY (numCommande),  
14    CONSTRAINT fk_cmd_client FOREIGN KEY (mailClient) REFERENCES CLIENT(mailClient),  
15    CONSTRAINT fk_cmd_localite FOREIGN KEY (codePostalLivraison) REFERENCES LOCALITE(codePostal),  
16  
17    CONSTRAINT ck_dates_livraison CHECK (dateLivraisonEffective >= dateCommande),  
18    CONSTRAINT ck_dates_realisation CHECK (dateRealisation >= dateCommande),  
19    CONSTRAINT ck_paiement CHECK (datePaiement >= dateCommande)  
20 );  
21
```

Détail des produits (Bouquets et Compositions)

Création de la table BOUQUET

Table BOUQUET Cette table correspond à l'entité faible **BOUQUET**. Elle est identifiée relativement à la commande (le bouquet n°1 de la commande X). L'attribut nbrBouquets permet de gérer la quantité de bouquets identiques commandés sans dupliquer les lignes de composition. Bien que dans le MCD, l'entité BOUQUET ne semble posséder que l'attribut idBouquet, elle est **identifiée relativement** à une commande (elle est "faible"). En SQL, cela oblige à intégrer l'identifiant du parent Commande dans la clé primaire de l'enfant.

```
1 CREATE TABLE BOUQUET (  
2     numCommande NUMBER(10),  
3     idBouquet NUMBER(5),  
4     nbrBouquets NUMBER(5) DEFAULT 1 NOT NULL,  
5  
6     CONSTRAINT pk_bouquet PRIMARY KEY (numCommande, idBouquet),  
7     CONSTRAINT fk_bouquet_cmd FOREIGN KEY (numCommande) REFERENCES COMMANDE(numCommande)  
8 );  
9
```

Création de la table COMPOSITION

Table COMPOSITION Cette table est la transformation de l'association **COMPOSER** reliant un bouquet à des fleurs. Une contrainte d'intégrité (quantite > 0) a été mise en place pour s'assurer qu'une ligne de composition contient nécessairement au moins une tige de fleur. Il est impossible de composer un bouquet avec une quantité de fleurs nulle ou négative. Sa clé primaire est constituée de l'ensemble des identifiants des entités qu'elle relie Commande, Bouquet et Fleur, cette clé composite garantit qu'on ne peut pas avoir deux fois la même fleur listée pour le même bouquet.

```
1 CREATE TABLE COMPOSITION (  
2     numCommande NUMBER(10),  
3     idBouquet NUMBER(5),  
4     nomFleur VARCHAR2(50),  
5     quantite NUMBER(5) NOT NULL,  
6  
7     CONSTRAINT pk_composition PRIMARY KEY (numCommande, idBouquet, nomFleur),  
8     CONSTRAINT fk_compo_bouquet FOREIGN KEY (numCommande, idBouquet) REFERENCES BOUQUET(numCommande, idBouquet),  
9     CONSTRAINT fk_compo_fleur FOREIGN KEY (nomFleur) REFERENCES FLEUR(nomFleur),  
10    CONSTRAINT ck_qte_fleur_pos CHECK (quantite > 0)  
11 );
```

Création de la table ACHAT

Table ACHAT Cette table résulte de la transformation de l'association **ACHETER** entre les entités Grossiste et Fleur. La clé primaire est composée du triplet (Grossiste, Fleur, Date) pour gérer l'historique. Des contraintes ont été ajoutées pour assurer que le prix d'achat unitaire et le nombre de fleurs achetées sont des valeurs strictement positives ou nulles (pour le prix), interdisant ainsi les erreurs de saisie comptable.

```
1 CREATE TABLE ACHAT (  
2     numSIREN VARCHAR2(20),  
3     nomFleur VARCHAR2(50),  
4     dateAchat DATE,  
5     prixAchatUnitaire NUMBER(5,2) NOT NULL,  
6     nbrFleurs NUMBER(5),  
7     CONSTRAINT pk_achat PRIMARY KEY (numSIREN, nomFleur, dateAchat),  
8     CONSTRAINT fk_achat_grossiste FOREIGN KEY (numSIREN) REFERENCES GROSSISTE(numSIREN),  
9     CONSTRAINT fk_achat_fleur FOREIGN KEY (nomFleur) REFERENCES FLEUR(nomFleur)  
10 );
```

Insertion des Données(Q5)

Insertion des données de référence (Localité et Fleurs) Avant de pouvoir créer une commande, il faut d'abord enregistrer la ville de livraison (pour le tarif) et les types de fleurs disponibles (pour le prix), afin de respecter les contraintes de clés étrangères de la base.



```
1  INSERT INTO LOCALITE (codePostal, ville, tarifLivraison)
2  VALUES ('69100', 'Villeurbanne', 6.00);
3
4  INSERT INTO FLEUR (nomFleur, tarifFleur)
5  VALUES ('Rose', 2.00);
6  INSERT INTO FLEUR (nomFleur, tarifFleur)
7  VALUES ('Tulipe', 1.50);
```

Création du profil de la cliente On enregistre ensuite Mme Juliette dans la base de données avec ses coordonnées complètes, son adresse email servant d'identifiant unique.



```
1  INSERT INTO CLIENT (
2      mailClient, nomClient,
3      adresseClient, codePostalClient,
4      villeClient, numTelClient
5  )
6  VALUES (
7      'juliette@laposte.net', 'Juliette',
8      '7, rue du général Leclerc', '69100',
9      'Villeurbanne', '0607890123'
10 );
11
```

Enregistrement de la commande générale On crée la commande liée à Mme Juliette. On y renseigne la date de la commande (01/10), la date de livraison prévue (05/10) ainsi que les coordonnées du destinataire, M. Jean. Notez que la date de paiement est renseignée car elle a réglé le jour même.

```
1 INSERT INTO COMMANDE (  
2     numCommande, mailClient,  
3     dateCommande, datePaiement,  
4     dateRealisation, dateLivraisonEffective,  
5     nomDestinataire, villeDestinataire,  
6     adresseDestinataire, telDestinataire,  
7     codePostalLivraison  
8 )  
9 VALUES (  
10     '1', 'juliette@laposte.net',  
11     TO_DATE('01/10/2025', 'DD/MM/YYYY'), TO_DATE('01/10/2025', 'DD/MM/YYYY'),  
12     TO_DATE('04/10/2025', 'DD/MM/YYYY'), NULL,  
13     'M. Jean', 'Villeurbanne',  
14     '60 grande rue', '0789012345',  
15     '69100'  
16 );
```

Définition du contenu de la commande (Les Bouquets) On précise maintenant que cette commande numéro 1 contient un type de bouquet spécifique (identifié par l'ID 1) et que la cliente en a commandé 2 exemplaires.

```
1 INSERT INTO BOUQUET (numCommande, numBouquet, nbrBouquets)  
2 VALUES (1, 1, 2);
```

Détail de la composition des bouquets Enfin, on décrit de quoi est constitué ce bouquet n°1 : on insère une ligne pour les 10 Roses et une autre ligne pour les 6 Tulipes.

```
1 INSERT INTO COMPOSITION (numCommande, numBouquet, nomFleur, quantite)  
2 VALUES (1, 1, 'Rose', 10);  
3  
4  
5 INSERT INTO COMPOSITION (numCommande, numBouquet, nomFleur, quantite)  
6 VALUES (1, 1, 'Tulipe', 6);  
7
```

Exploitation des Données(Q7)

R1. Le nombre de commandes payées en 2021

Afin d'évaluer le volume d'activité sur l'année de référence, cette requête comptabilise le nombre total de commandes ayant été effectivement réglées au cours de l'année 2021.



```
1 SELECT COUNT(*) AS NbCommandesPayees2021
2 FROM COMMANDE
3 WHERE datePaieement IS NOT NULL
4 AND TO_CHAR(datePaieement, 'YYYY') = '2021';
```

	NBCOMMANDESPAYEES2021
1	127

R2. Les 3 premiers codes postaux où il y a le plus de livraisons

Pour identifier les zones géographiques prioritaires, nous avons extrait les trois codes postaux de destination les plus fréquents, permettant ainsi de cibler les secteurs où la demande est la plus forte.



```
1 SELECT codePostalLivraison, COUNT(*) AS NbLivraisons
2 FROM COMMANDE
3 WHERE codePostalLivraison IS NOT NULL
4 GROUP BY codePostalLivraison
5 ORDER BY NbLivraisons DESC
6 FETCH FIRST 3 ROWS ONLY;
```

	CODEPOSTALLIVRAISON	NBLIVRAISONS
1	69230	27
2	69680	24
3	69200	21

R3. La fleur la plus utilisée en 2021

Cette analyse permet de déterminer la variété de fleur la plus populaire de l'année 2021, en calculant le volume total de tiges utilisées dans l'ensemble des compositions commandées.



```
1  SELECT c.nomFleur, SUM(c.quantite * b.nbrBouquets) AS
   TotalUtilise
2  FROM COMPOSITION c
3  JOIN BOUQUET b ON c.numCommande = b.numCommande AND c.
   numBouquet = b.numBouquet
4  JOIN COMMANDE cmd ON b.numCommande = cmd.numCommande
5  WHERE TO_CHAR(cmd.dateCommande, 'YYYY') = '2021'
6  GROUP BY c.nomFleur
7  ORDER BY TotalUtilise DESC
8  FETCH FIRST 1 ROWS ONLY;
```

	NOMFLEUR	TOTALUTILISE
1	Tournesol	438

R4. Le nombre moyen de bouquets par client

Dans le but de mieux cerner le panier moyen et les habitudes de consommation, nous calculons ici la moyenne du nombre de bouquets achetés par client.



```
1  SELECT AVG(TotalBouquets) AS MoyenneBouquetsParClient
2  FROM (
3      SELECT cmd.mailClient, SUM(b.nbrBouquets) AS Total
   Bouquets
4      FROM COMMANDE cmd
5      JOIN BOUQUET b ON cmd.numCommande = b.numCommande
6      GROUP BY cmd.mailClient
7  );
```

	MOYENNEBOUQUETSPARCLIENT
1	17,8

R5. Les fleurs jamais utilisées dans les bouquets

Cette requête d'audit permet d'identifier les stocks dormants en listant les variétés de fleurs présentes au catalogue mais qui n'ont, à ce jour, jamais été intégrées dans une composition vendue.



```
1 SELECT nomFleur
2 FROM FLEUR
3 WHERE nomFleur NOT IN (SELECT DISTINCT nomFleur FROM COMPOSITION);
```

	NOMFLEUR
1	Marguerite
2	Magnolia
3	Iris
4	Pensee

R6. La fleur préférée par client

Dans une optique de gestion de la relation client (CRM), cette requête détermine pour chaque acheteur la variété de fleur qu'il commande le plus souvent, ce qui permettrait de personnaliser de futures offres.



```
1 SELECT mailClient, nomFleur
2 FROM (
3     SELECT cmd.mailClient, c.nomFleur,
4           SUM(c.quantite * b.nbrBouquets) AS QteTotale,
5           RANK() OVER (PARTITION BY cmd.mailClient ORDER BY SUM(c.quantite * b.nbrBouquets) DESC) as Rang
6     FROM COMMANDE cmd
7     JOIN BOUQUET b ON cmd.numCommande = b.numCommande
8     JOIN COMPOSITION c ON b.numCommande = c.numCommande AND b.numBouquet = c.numBouquet
9     GROUP BY cmd.mailClient, c.nomFleur
10 )
11 WHERE Rang = 1;
12
```

MAILCLIENT	NOMFLEUR
1 artisan_des_saveurs@yahoo.com	Narcisse
2 atelier_de_luxe@protonmail.com	Edelweiss
3 atelier_de_luxe@protonmail.com	Hortensia
4 aubry@orange.fr	Anemone
5 barbot@outlook.com	Jasmin
6 baron@laposte.net	Oeillet
7 bazar_et_creation@sfr.fr	Camelia
8 benedetti@hotmail.com	Tiare
9 benoit@laposte.net	Chrysantheme
10 benoit@protonmail.com	Camelia
11 berger@outlook.com	Edelweiss
12 bernard@sfr.fr	Camelia
13 bertin@laposte.net	Glaieul

R7. La liste des numéros de commande associés à la date, au nom du client et au montant

Voici un état récapitulatif complet des commandes. Cette requête complexe agrège les coûts des fleurs (quantité × prix unitaire) et y ajoute les frais de livraison spécifiques à la localité pour obtenir le montant total facturé.

```
1 SELECT cmd.numCommande, cmd.dateCommande, cl.nomClient,  
2         (SUM(f.tariffFleur * c.quantite * b.nbrBouquets) + NVL(l.tarifLivraison, 0)) AS MontantTotal  
3 FROM COMMANDE cmd  
4 JOIN CLIENT cl ON cmd.mailClient = cl.mailClient  
5 JOIN BOUQUET b ON cmd.numCommande = b.numCommande  
6 JOIN COMPOSITION c ON b.numCommande = c.numCommande AND b.numBouquet = c.numBouquet  
7 JOIN FLEUR f ON c.nomFleur = f.nomFleur  
8 LEFT JOIN LOCALITE l ON cmd.codePostalLivraison = l.codePostal  
9 GROUP BY cmd.numCommande, cmd.dateCommande, cl.nomClient, l.tarifLivraison;  
10
```

	NUMCOMMANDE	DATECOMMANDE	NOMCLIENT	MONTANTTOTAL
1	227	02/12/22	Restaurant Mets Épicuriens	183
2	234	07/12/22	Chauvin	85
3	245	12/12/22	Boutique Bazar et Création	264
4	248	12/12/22	Boutique Atelier de Luxe	190,3
5	256	14/12/22	Boutique Tendance et Cie	127
6	259	15/12/22	Boutique Magasin Secret	208,8
7	272	20/12/22	Montel	278,8
8	278	23/12/22	Boutique Esprit Bohème	288,5
9	307	09/06/23	Restaurant Comptoir du Terroir	361,2
10	309	12/06/23	Rogers	388,7
11	312	14/06/23	Boutique Magasin Secret	277,8
12	316	14/06/23	Clerc	69
13	319	15/06/23	Clerc	63,5

R8. Le nom du client qui a passé la commande la plus chère avec retrait en magasin

Nous recherchons ici le client ayant effectué la transaction la plus importante parmi celles ne nécessitant pas de livraison (retrait boutique), ce qui exclut les frais de port du calcul.

```
1 SELECT cl.nomClient  
2 FROM COMMANDE cmd  
3 JOIN CLIENT cl ON cmd.mailClient = cl.mailClient  
4 JOIN BOUQUET b ON cmd.numCommande = b.numCommande  
5 JOIN COMPOSITION c ON b.numCommande = c.numCommande AND b.numBouquet = c.numBouquet  
6 JOIN FLEUR f ON c.nomFleur = f.nomFleur  
7 WHERE cmd.codePostalLivraison IS NULL  
8 GROUP BY cmd.numCommande, cl.nomClient  
9 ORDER BY SUM(f.tariffFleur * c.quantite * b.nbrBouquets) DESC  
10 FETCH FIRST 1 ROWS ONLY;
```

	NOMCLIENT
1	Boutique Esprit Bohème

R9. Le nom du client qui a passé la commande la plus chère (Livraison ou Retrait)

Enfin, cette requête identifie le 'record' de vente du magasin en affichant le client à l'origine de la commande la plus onéreuse enregistrée, tous modes de mise à disposition confondus.

```
1 SELECT cl.nomClient
2 FROM COMMANDE cmd
3 JOIN CLIENT cl ON cmd.mailClient = cl.mailClient
4 JOIN BOUQUET b ON cmd.numCommande = b.numCommande
5 JOIN COMPOSITION c ON b.numCommande = c.numCommande AND b.numBouquet = c.numBouquet
6 JOIN FLEUR f ON c.nomFleur = f.nomFleur
7 LEFT JOIN LOCALITE l ON cmd.codePostalLivraison = l.codePostal
8 GROUP BY cmd.numCommande, cl.nomClient, l.tarifLivraison
9 ORDER BY (SUM(f.tarifFleur * c.quantite * b.nbrBouquets) + NVL(l.tarifLivraison, 0)) DESC
10 FETCH FIRST 1 ROWS ONLY;
```

NOMCLIENT
1 Moreau

[Lien vers les codes utilisés](#)